```
from __future__ import annotations

import json
import logging
from pathlib import Path
from typing import Optional, Dict, Any

from agent1_system_guardian import SystemGuardian
from agent2_task_executor import TaskExecutor
from agent3_planner import Planner
from agent4_reward_memory import RewardMemory
from agent6_interface_manager import InterfaceManager
from agent007_system_introspector import SystemIntrospector007
from app_watcher import AppWatcher
from app_classifier import AppClassifier
from dgm_engine import SafeDGM
from voice_interface import VoiceInterface
from visual_observer import VisualObserver
from data_structures import GuardianAlert
from autonomy_policy import AutonomyPolicy
from agent_eagle_eyes0 import EagleEyes0


class LucyAssistant:
    def __init__(self, base_dir: Path) -> None:
        self.base_dir = Path(base_dir)
        # ... existing code ...

        # Initialize planner
        self.planner = Planner()

        # Eagle Eyes — code debugger and crash analyst
        self.eagle = EagleEyes0(
            base_dir=self.base_dir,
            task_executor=self.task_executor,
            reward_memory=self.reward_memory,
        )

        # Updated InterfaceManager constructor
        self.interface = InterfaceManager(
            planner=self.planner,
            task_executor=self.task_executor,
            autonomy_policy=self.policy,
            eagle_eye=self.eagle,
```

```
        reward_memory=self.reward_memory,
    )
    # ... rest of initialization code ...

def on_guardian_alert(self, alert: GuardianAlert) -> None:
    try:
        # ... existing code ...

        alert_text = ...  # ← presumably created earlier in the method

        reply = self.interface.handle_user_text(alert_text)

        print("\n--- Lucy automatic plan ---")
        print(reply)
        print("---------------------------\n")

    except Exception as e:
        # ... existing error handling ...

def on_new_process(self, proc_info: Dict[str, Any]) -> None:
    try:
        # ... existing code ...

        prompt = ...  # ← presumably created earlier

        reply = self.interface.handle_user_text(prompt)

        print("\n--- Lucy app insight ---")
        print(reply)
        print("------------------------\n")

    except Exception as e:
        # ... existing error handling ...

def run_cli(self) -> None:
    # ... existing setup code ...

    while True:
        # ... input handling ...

        user_text = input("You: ").strip()   # example — adapt to your actual input method

        if not user_text:
            continue
```

```
            reply = self.interface.handle_user_text(user_text)
            print(reply)


# Optional: small improvement suggestion for run_cli
# (uncomment if it fits your style)

#     def run_cli(self) -> None:
#         print("Lucy CLI started. Type 'exit' or Ctrl+C to quit.\n")
#
#         while True:
#             try:
#                 user_text = input("You: ").strip()
#                 if user_text.lower() in ('exit', 'quit', '/q'):
#                     print("Goodbye.")
#                     break
#                 if not user_text:
#                     continue
#
#                 reply = self.interface.handle_user_text(user_text)
#                 print(reply)
#                 print()
#
#             except KeyboardInterrupt:
#                 print("\nGoodbye.")
#                 break
#             except Exception as e:
#                 print(f"Error: {e}")
```